API & REST API

Complete one-page reference · HTTP Methods · Status Codes

Quick concepts

Application Programming Interface

Representational State Transfer

HTTP/1.1 · JSON · CRUD

What is an API?

An **API (Application Programming Interface)** is a contract that lets two pieces of software talk to each other. One side makes a *request*, the other side returns a *response*.

Analogy: a waiter (API) takes your order (request) to the kitchen (server) and brings back your food (response).

Key parts of every API call:

- **Endpoint** — the URL address of the resource
- **Method** — the action (GET, POST, …)
- **Headers** — metadata (auth token, content type)
- **Body** — data sent with the request (POST/PUT/PATCH)
- **Response** — status code + JSON data returned

What is REST?

REST (Representational State Transfer) is an architectural style for designing APIs over HTTP. It is not a protocol — it is a set of six constraints.

REST principles:

- **Stateless** — every request is self-contained, no server sessions
- **Resource-based URLs** — nouns, not verbs: `/users/42`
- **Uniform interface** — HTTP verbs define the action
- **Client–server** — UI and data storage are separated
- **Cacheable** — responses can declare if they are cacheable
- **Layered system** — load balancers, gateways are transparent

REQUEST LIFECYCLE

Client → HTTP Request → API Gateway → Server Logic → Database → ← JSON Response

HTTP METHODS (VERBS)

GET Read / Retrieve

Fetches a resource or list of resources. Never modifies data. Safe and idempotent — calling it 100 times has the same result as calling it once.

```
GET /users/42

GET /users?page=1&limit;=20
```

Body: None · Idempotent: Yes · Safe: Yes · Status: 200 OK

POST Create

Submits data to create a new resource. Not idempotent — calling it twice creates two separate records. The server assigns the new resource's ID.

```
POST /users

{ "name": "Layla", "email": "layla@co.com"
}
```

Body: Required · Idempotent: No · Safe: No · Status: 201 Created

PUT Replace (full update)

Replaces an entire resource with the supplied data. If a field is omitted, it is removed. Idempotent — sending the same request twice yields the same result.

```
PUT /users/42

{ "name": "Layla", "email":
"layla@co.com", "age": 28 }
```

Body: Required · Idempotent: Yes · Safe: No · Status: 200 / 204

PATCH Partial update

Updates only the specified fields of a resource. More efficient than PUT when you need to change just one property. Generally idempotent in practice.

```
PATCH /users/42

{ "age": 29 }
```

Body: Required · Idempotent: Usually · Safe: No · Status: 200 / 204

DELETE Remove

Permanently deletes a resource. Idempotent — deleting something that no longer exists still returns success (204). Usually returns no body.

```
DELETE /users/42
```

Body: None · Idempotent: Yes · Safe: No · Status: 204 No Content

HEAD Headers only

Identical to GET but the server returns only headers, no body. Used to check if a resource exists, get its size, or validate a cache without downloading data.

```
HEAD /files/report.pdf
```

Body: None · Idempotent: Yes · Safe: Yes · Status: 200 OK

OPTIONS Capabilities / CORS preflight

Returns the HTTP methods an endpoint supports. Browsers send this automatically before cross-origin requests (CORS preflight) to check server permissions.

```
OPTIONS /users → Allow: GET, POST, HEAD
```

Body: None · Idempotent: Yes · Safe: Yes · Status: 200 + Allow header

CRUD ↔ HTTP METHOD MAPPING

CRUD Operation	HTTP Method	Typical URL	Success Status	Idempotent
Create	POST	/users	201 Created	No
Read (list)	GET	/users	200 OK	Yes
Read (one)	GET	/users/42	200 OK	Yes
Update (full)	PUT	/users/42	200 / 204	Yes
Update (part)	PATCH	/users/42	200 / 204	Usually
Delete	DELETE	/users/42	204 No Content	Yes

HTTP STATUS CODES

2xx Success

200	OK — request succeeded
201	Created — resource created
204	No Content — success, no body

3xx Redirect

301	Moved Permanently
304	Not Modified — use cache

4xx Client Error

400	Bad Request — invalid syntax
401	Unauthorized — auth required
403	Forbidden — no permission
404	Not Found — resource missing
409	Conflict — state conflict
429	Too Many Requests — rate limited

5xx Server Error

500	Internal Server Error
502	Bad Gateway
503	Service Unavailable

REST URL design rules

Use nouns, not verbs	`/users/42` not `/getUser`
Plural collections	`/users` not `/user`
Nest related resources	`/users/42/posts`
Filter via query params	`/users?role=admin&active;=true`
Lowercase, hyphens	`/blog-posts` not `/blogPosts`
API versioning in path	`/v1/users` or `/v2/users`

EXAMPLE: FULL HTTP REQUEST & RESPONSE

→ Request

```
POST /v1/users HTTP/1.1
Host: api.example.com
Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json
Accept: application/json

{ "name": "Layla Hassan", "email":
"layla@example.com", "role": "admin" }
```

← Response

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /v1/users/99

{
"id": 99,
"name": "Layla Hassan",
"email": "layla@example.com",
"role": "admin",
"createdAt": "2025-05-10T14:32:00Z"
}
```